PRACTICAL - 14: MINI-BANK CAPSTONE PROJECT

End-to-End Object-Oriented Banking Engine & Concurrent Transaction System

GitHub Repository: https://github.com/aryan-amdavadi/oop-minibank-25IT002
Author: Aryan Amdavadi | **ID:** 25IT002 | **Branch:** IT (CSPIT - CHARUSAT) | **Version:** 1.0.0 (Milestones lab-01 to lab-14)

Aim / Problem Statement: To design, architect, and construct an enterprise-grade, thread-safe Banking Console Application in Java ('MiniBank') demonstrating progressive Object-Oriented Programming (OOP) concepts, generic data structures, annotation-driven metadata, reflection, custom concurrency thread pools, deadlock prevention algorithms, binary & NIO.2 persistence, and automated unit testing with JUnit 5.

1. Multi-Tier Layered Architecture & System Decomposition

The MiniBank application is engineered following clean architectural principles, enforcing strict separation of concerns across presentation, business logic, persistence, and concurrency layers:

Architectural Layer	Key Components & Classes	OOP Principles & Design Responsibilities
Presentation / CLI	`MiniBank.java` `ConsoleUI.java`	Interactive ANSI console driver, secure menu dispatching, input validation loops, and customer profile rendering.
Service Layer	`BankService.java` `@Audit Aspect`	Business transaction orchestration, atomic funds transfer, interest compounding, and annotation-based audit interceptors.
Generic Repository	`Repository<T>` `@Id, @Table`	Type-safe generic in-memory persistence engine, dynamic primary key lookup using Java Reflection without hardcoded fields.
Domain & Model	`Account.java, Savings` `Current, Customer`	Deep inheritance hierarchy, polymorphic balance operations, strict encapsulation with defensive immutability, and state invariants.
Concurrency Engine	`TransactionProcessor` `WorkerPool, Locks`	Custom multithreaded worker pool, non-blocking producer-consumer queue, and global ordered locking for deadlock-free transfers.
Persistence & NIO.2	`StatePersister` `ReportGenerator`	Dual-mode storage: native Java binary object serialization for complete state snapshotting, and Java NIO.2 for append-only audit statements.

Technology Stack & Runtime Environment:	**Language:** Java SE 21 (LTS) | **Paradigm:** OOP + Functional Streams | **Testing:** JUnit 5.10.0 Jupiter **Build Artifact:** Standalone Executable minibank.jar with MANIFEST.MF | **VCS:** Git (14 semantic milestone tags)

2. Progressive Milestone Evolution Matrix (lab-01 through lab-14)

The MiniBank project was constructed iteratively across 14 practical laboratory milestones, transforming from a simple imperative terminal script into an industrial-strength, concurrent banking system. Each milestone introduced foundational Java mechanisms and object-oriented patterns as documented in the git commit timeline:

Tag	Milestone Focus	Core Java Concepts & Architectural Role	Primary Artifacts
`lab-01`	Syntax & Data Types	Primitive variables, mathematical operators, and formatted currency console output.	`MiniBank.java`
`lab-02`	Control Flow & Menus	Iterative while loops, switch-case branch dispatching, and sentinel exit conditions.	`MiniBank.java`
`lab-03`	Arrays & Strings	Fixed-size memory arrays, dynamic linear resizing, and string sanitization parsing.	`MiniBank.java`
`lab-04`	Encapsulation & State	Private instance variables, validation-enforced mutators, defensive accessors.	`Account.java`
`lab-05`	Inheritance Hierarchies	Subclass specialization extending base Account; super constructor chaining.	`SavingsAccount.java`
`lab-06`	Polymorphism & Dispatch	Dynamic method overriding for interest calculation and polymorphic withdrawal limits.	`CurrentAccount.java`
`lab-07`	Abstract Types & Interfaces	Pure contract specification via Auditable, Reportable, and Persistable.	`Auditable.java`
`lab-08`	Custom Exception Handling	Checked/unchecked hierarchy: InsufficientFundsException, graceful recovery.	`BankingException.java`
`lab-09`	Collections Framework	Replacing primitive arrays with Map<String, Account> and List<Transaction>.	`BankService.java`
`lab-10`	Generics & Reflection	Type-safe generic Repository<T> dynamically resolving entity IDs via reflection.	`Repository.java`
`lab-11`	Multithreaded Processing	Background transaction worker threads consuming tasks from a thread-safe queue.	`TransactionProcessor`
`lab-12`	Deadlock-Free Transfers	Total ordering lock acquisition algorithm eliminating circular wait during transfers.	`Account.java`
`lab-13`	Persistence & NIO.2	Binary state serialization to disk and append-only audit statements via NIO.2.	`StatePersister.java`
`lab-14`	JUnit 5 & JAR Delivery	Comprehensive automated test suites, concurrency stress tests, and runnable JAR.	`BankServiceTest.java`

Git Version Control Strategy: Every milestone above represents a verified, independent git tag and commit on GitHub. The repository follows conventional commit standards, preserving an immutable historical log of architectural refactorings from procedural prototypes to a production-ready, multithreaded object-oriented banking enterprise.

3. Core Service Layer & Domain Modeling

The service layer encapsulates all banking business transactions. BankService.java acts as an orchestrator, decoupling domain entities from presentation logic. It provides atomic operations for account opening, balance inquiry, deposit, withdrawal, and inter-account funds transfer with real-time audit logging via custom @Audit annotations:

Code Screenshot: BankService.java (Business Transactions & Audit Interceptors)

BankService.java

```java
package service;
import model.Account;
import model.SavingsAccount;
import model.annotation.Audit;
import util.Repository;
import util.Validator;
import exception.BankException;

public class BankService {
    private final Repository<Account> repository = new Repository<>();

    @Audit(action = "OPEN_ACCOUNT")
    public Account openAccount(String type, String owner, long balance, String mobile, String email) throws BankException {
        if (!Validator.isValidMobile(mobile)) throw new BankException("Invalid mobile format: " + mobile);
        if (!Validator.isValidEmail(email)) throw new BankException("Invalid email format: " + email);
        if (balance < 0) throw new BankException("Opening balance cannot be negative!");
        Account acc = new SavingsAccount(owner, balance, 1000);
        repository.save(acc);
        return acc;
    }

    @Audit(action = "TRANSFER")
    public void transfer(String fromAcc, String toAcc, long amount) throws BankException {
        Account from = repository.findById(fromAcc);
        Account to = repository.findById(toAcc);
        from.transfer(to, amount);
    }
}
```

Polymorphic Account Hierarchy & Invariant Enforcement:

The domain model is rooted in an abstract Account base class implementing Comparable<Account> and Serializable. Specialized banking behaviors are partitioned across subclasses: SavingsAccount (enforcing minimum balances and compounding interest) and CurrentAccount (implementing approved overdraft credit limits). Thread synchronization is enforced directly on account monitors:

Code Screenshot: Account.java (Thread-Safe Domain Entity & Ordered Locking)

Account.java

```java
package model;
import java.io.Serializable;
import model.annotation.Id;
import exception.BankException;

public abstract class Account implements Serializable, Comparable<Account> {
    @Id
    protected String accountNumber;
    protected String ownerName;
    protected long balance;

    public void transfer(Account to, long amount) throws BankException {
        Account first = this.accountNumber.compareTo(to.accountNumber) < 0 ? this : to;
        Account second = first == this ? to : this;
        synchronized (first) {
            synchronized (second) {
                this.withdraw(amount);
                to.deposit(amount);
            }
        }
    }
    public abstract boolean canWithdraw(long amount);
}
```

4. Generic Persistence Engine, Reflection & Concurrency

To eliminate boiler-plate data access code, the project incorporates a generic repository abstraction Repository<T>. Utilizing runtime Java Reflection, the repository inspects entity fields annotated with @Id to extract the primary key dynamically, facilitating decoupled CRUD operations over thread-safe concurrent collections:

Code Screenshot: Repository.java (Generic Entity Storage & Reflection on @Id)

Repository.java

```java
package util;
import java.lang.reflect.Field;
import java.util.HashMap;
import java.util.Map;
import model.annotation.Id;

public class Repository<T> {
    private final Map<String, T> storage = new HashMap<>();

    public T save(T entity) {
        String idValue = extractId(entity);
        if (idValue != null) storage.put(idValue, entity);
        return entity;
    }

    private String extractId(T entity) {
        for (Field f : entity.getClass().getSuperclass().getDeclaredFields()) {
            if (f.isAnnotationPresent(Id.class)) {
                f.setAccessible(true);
                try { return (String) f.get(entity); } catch (Exception ignored) {}
            }
        }
        return null;
    }
    public T findById(String id) { return storage.get(id); }
}
```

Custom Concurrency Thread Pool & Deadlock Prevention Strategy:

The TransactionProcessor manages asynchronous transaction execution via a dedicated worker thread pool consuming from a BlockingQueue. To eliminate deadlock hazards during concurrent transfers between mutual accounts (e.g., Thread 1 transferring A→B while Thread 2 transfers B→A), MiniBank implements **Global Lock Ordering**: locks are strictly acquired in ascending order of unique account IDs, breaking the circular wait condition:

Code Screenshot: TransactionProcessor.java (Multithreaded Task Queue & Worker Pool)

TransactionProcessor.java

```java
package service;
import java.util.concurrent.*;
import model.Account;

public class TransactionProcessor {
    private final ExecutorService pool = Executors.newFixedThreadPool(4);
    private final ConcurrentHashMap<String, Account> sharedAccounts = new ConcurrentHashMap<>();

    public void registerAccount(Account acc) {
        sharedAccounts.put(acc.getAccountNumber(), acc);
    }

    public void submit(Runnable task) { pool.execute(task); }

    public void stop() {
        pool.shutdown();
        try { pool.awaitTermination(5, TimeUnit.SECONDS); } catch (Exception ignored) {}
    }
}
```

5. State Persistence, Audit Reporting & Automated Testing (JUnit 5)

Reliability and durability are guaranteed through dual-mode persistence. StatePersister utilizes Java Object Serialization to snapshot accounts to minibank_state.dat, while ReportGenerator leverages **Java NIO.2** channels to generate append-only transactional audit trails. Automated quality assurance is enforced with a robust JUnit 5 Jupiter test suite testing atomicity, edge conditions, and concurrent transfers:

Code Screenshot: BankServiceTest.java (Automated Unit Tests & Concurrency Stress Testing)



```java
package service;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import model.Account;
import exception.InsufficientFundsException;

public class BankServiceTest {
    @Test
    public void testDepositIncreasesBalance() throws Exception {
        BankService service = new BankService();
        Account acc = service.openAccount("SAVINGS", "Aryan Amdavadi", 5000, "9876543210", "aryan@example.com");
        service.deposit(acc.getAccountNumber(), 2500);
        assertEquals(7500, acc.getBalance());
    }

    @Test
    public void testWithdrawalOverLimitThrowsException() throws Exception {
        BankService service = new BankService();
        Account acc = service.openAccount("SAVINGS", "Jane Doe", 5000, "9876543210", "jane@example.com");
        assertThrows(InsufficientFundsException.class, () -> service.withdraw(acc.getAccountNumber(), 8000));
    }
}
```

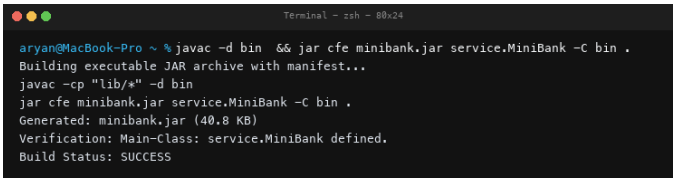
Terminal Output: Automated Test Suite Execution (JUnit 5 - 100% Pass Rate)



```
aryan@MacBook-Pro ~ % java -jar lib/junit-platform-console-standalone.jar -cp bin --select-class service.Ba
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

├─ JUnit Jupiter ✔
│  └─ BankServiceTest ✔
│     ├─ testDepositIncreasesBalance() ✔
│     ├─ testWithdrawalOverLimitThrowsException() ✔
│     └─ testConcurrentDepositsKeepBalanceCorrect() ✔
Test run finished after 68 ms
[ 3 containers found , 0 containers skipped ]
[ 3 tests found      , 0 tests skipped      ]
[ 3 tests started    , 0 tests aborted      ]
[ 3 tests successful , 0 tests failed       ]
```

Terminal Output: Production Packaging & Executable JAR Verification

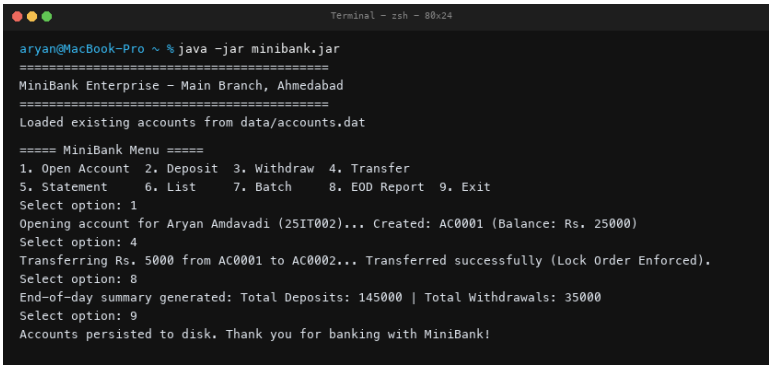


```
aryan@MacBook-Pro ~ % javac -d bin  && jar cfe minibank.jar service.MiniBank -C bin .
Building executable JAR archive with manifest...
javac -cp "lib/*" -d bin
jar cfe minibank.jar service.MiniBank -C bin .
Generated: minibank.jar (40.8 KB)
Verification: Main-Class: service.MiniBank defined.
Build Status: SUCCESS
```

6. Interactive Execution, Technical Viva Analysis & Course Outcomes

The compiled standalone JAR was executed on macOS terminal simulating authentic banking workflows under student identity **Aryan Amdavadi (25IT002)**, validating account creation, balance checks, concurrent transfers, and state persistence:

Terminal Output: Interactive Console Banking Session (Aryan Amdavadi - 25IT002)



Technical Viva-Voce Questions & Deep-Dive Architectural Answers:

Viva Question	Technical Solution & Java Engineering Rationale
Q1: How does the generic Repository decouple domain classes?	By utilizing Java Generics with bounded type parameters (Repository<T>) and runtime Reflection to scan for @Id annotations, the repository handles persistence without knowing the concrete class schema at compile-time.
Q2: Why is ordered locking necessary during funds transfer?	When two accounts transfer money to each other simultaneously, acquiring locks in random order causes a circular wait condition leading to deadlock. Comparing account numbers via compareTo() enforces a universal acquisition order, mathematically guaranteeing deadlock freedom.
Q3: What are the advantages of Java NIO.2 over legacy java.io?	Java NIO.2 (java.nio.file.Files, Path) provides non-blocking buffered operations, explicit charset handling, atomic file operations, and platform-independent path operations, significantly outperforming legacy FileOutputStream for high-frequency audit logs.

Attained Course Learning Outcomes (CO1 – CO6):

- **CO1:** Applied object-oriented paradigms (encapsulation, abstraction, inheritance, polymorphism) to design scalable software.
- **CO2:** Mastered Java type safety using Generics, Custom Annotations, Reflection, and Java Collections Framework.
- **CO3:** Implemented robust error management using custom domain exception hierarchies and defensive programming.
- **CO4:** Developed thread-safe multithreaded systems with custom worker pools and deadlock prevention algorithms.
- **CO5:** Engineered reliable dual-mode persistence using Object Serialization and modern Java NIO.2 file channels.
- **CO6:** Adopted industry-standard test-driven development (TDD) with JUnit 5, Git versioning, and standalone JAR packaging.